

CEN206 Object-Oriented Programming Syllabus

Detailed Syllabus

Author: Asst. Prof. Dr. Uğur CORUH

List of Figures

List of Tables

Recep Tayyip Erdogan University
Faculty of Engineering and Architecture
Computer Engineering
CEN206 Object-Oriented Programming
Syllabus

Spring Semester, 2025-2026

Download DOC¹, SLIDE²

Instructor	Asst. Prof. Dr. Uğur CORUH
Contact Information	ugur.coruh@erdogan.edu.tr
Office No	F-301
Google Classroom Code	N/A
Microsoft Teams Code	rf8tjxc
Lecture Hours and Days	Friday 13:00-16:00 (Theory and Lab) İİBF-402

Lecture Class-room	İİBF & Law Faculty Building / İİBF-402 or Online via Google Meet / Microsoft Teams
Office Hours	Meetings will be scheduled over Google Meet or Microsoft Teams with your university account and email and performed via demand emails. Please send emails with the subject starting with [CEN206] tag for the fast response and write formal, clear, and short emails

¹syllabus.en.md_doc.pdf

²syllabus.en.md_slide.pdf

Lecture and Communication	
Language	English
Theory/Laboratory Course Hour Per Week	3/0 Hours
Credit	3
Prerequisite	None
Corequisite	None
Requirement	None

A.Course Description

This course introduces advanced programming skills with a focus on object-oriented programming and design fundamentals using a high-level language such as Java. Object-oriented programming is the process of integrating software components into a large-scale software architecture. After learning the fundamentals of coding, this approach to software development is the next logical step, allowing for large-scale programs. The course focuses on understanding and applying object-oriented concepts such as classes, objects, data abstraction, methods, method overloading, inheritance, and polymorphism. The class will be built around sharing expertise and guiding students to find learning methods and practice for object-oriented programming topics. Making programming applications and projects in the courses will strengthen the learning process by putting theory into practice.

B.Course Learning Outcomes

After completing this course satisfactorily, a student will be able to:

ID	Course Learning Outcome
CLO.1	Explains the fundamental principles (encapsulation, inheritance, polymorphism, abstraction) and advantages of object-oriented programming.
CLO.2	Designs and implements classes (including constructors, destructors, access modifiers, static members), objects, and methods in a specific programming language (e.g., C++, Java, C#).
CLO.3	Applies inheritance (derived classes, method overriding) and polymorphism (virtual methods, abstract classes, interfaces) to ensure code reusability and flexibility.
CLO.4	Uses exception handling (try-catch) mechanisms to manage errors in programs.
CLO.5	Creates type-independent, reusable code components using Templates (C++) or Generics (Java/C#).
CLO.6	Performs basic file I/O operations and uses data structures from standard libraries (e.g., STL, Java Collections).
CLO.7	Develops medium-complexity, maintainable, and extensible software systems using object-oriented design principles (e.g., introduction to SOLID) and composition.

C.Course Topics

- Object-oriented programming fundamentals (Encapsulation, Inheritance, Polymorphism, Abstraction)
- Classes, objects, methods, constructors, access modifiers

- Interfaces, abstract classes, lambda expressions, generics
 - UML - Unified Modeling Language (Class, Sequence, Use Case, State Diagrams)
 - PlantUML - Text-based UML diagram generation
 - UMPLE - Model-Driven Development and code generation
 - GoF Design Patterns (Creational, Structural, Behavioral)
 - Code smells and refactoring techniques
 - SOLID principles introduction
-

D.Textbooks and Required Hardware or Equipment

This course does not require a coursebook. If necessary, you can use the following books and open-source online resources.

- *Timothy C. Lethbridge and Robert Laganière, Object-Oriented Software Engineering: Practical Software Development using UML and Java, McGraw Hill*
 - *Walter Savitch, Absolute C++, Addison-Wesley Longman*
-

- *Intro to Java Programming, Comprehensive Version (10th Edition) 10th Edition by Y. Daniel Liang*
 - *Harvey M. Deitel and Paul J. Deitel. 2001. Java How to Program (4th. ed.). Prentice Hall PTR, USA.*
 - *Paul Deitel and Harvey Deitel. 2016. Visual C# How to Program (6th. ed.). Pearson.*
 - *Additional Books TBD*
-

During this course, you should have a laptop for programming practices. You will have your development environment, and you will use this for examination and assignments also classroom practices.

E.Grading System

You will complete one project and two written quizzes throughout the semester. You are expected to submit your Midterm Project Report (MPR1) at the midterm, demonstrating your project progress and implementation. In the 15th week, you will present and submit your Final Project Report (MPR2).

You will take a written quiz (QUIZ1) in the 7th week and another (QUIZ2) in the 13th week.

Assessment	Code	Weight	Scope
Midterm Project Report	MPR1	60%	Midterm
Quiz-1	QUIZ1	40%	Midterm
Final Project Report	MPR2	70%	Final
Quiz-2	QUIZ2	30%	Final

$$Grade_{Midterm} = 0.6MPR1 + 0.4QUIZ1$$

$$GradeFinal = 0.7MPR2 + 0.3QUIZ2$$

$$PassingGrade = (40 * GradeMidterm + 60 * GradeFinal)/100$$

F. Instructional Strategies and Methods

The basic teaching method of this course will be planned to be face-to-face in the classroom, and support resources, home works, and announcements will be shared over google classroom. In unexpected situations course will be planned for online for disaster scenarios. Students are expected to be in the university if face-to-face method selected. This responsibility is very important to complete this course with success. If pandemic situation changes and distance education is required during this course, this course will be done using synchronous and asynchronous distance education methods. In this scenario, students are expected to be in the online platform, zoom, or meet at the time specified in the course schedule. Attendance will be taken.

G. Late Homework

Throughout the semester, assignments must be submitted as specified by the announced deadline. Overdue assignments will not be accepted. Unexpected situations must be reported to the instructor for late homeworks by students

H. Course Platform and Communication

Google Classroom and Github will be used as a course learning management system. All electronic resources and announcements about the course will be shared on this platform. It is very important to check the course page daily, access the necessary resources and announcements, and communicate with the instructor to complete the course with success.

I. Academic Integrity, Plagiarism & Cheating

Academic Integrity is one of the most important principles of RTEÜ University. Anyone who breaches the principles of academic honesty is severely punished.

It is natural to interact with classmates and others t.”study together”. It may also be the case where a student asks to help from someone else, paid or unpaid, better understand a difficult topic or a whole course. However, what is the borderline between “studying together” or “taking private lessons” and “academic dishonesty”? When is it plagiarism, when is it cheating?

It is obvious that looking at another student’s paper or any source other than what is allowed during the exam is cheating and will be punished. However, it is known that many students come to university with very little experience concerning what is acceptable and what counts as “copying,” especially for assignments.

The following are attempted as guidelines for the Faculty of Engineering and Architecture students to highlight the philosophy of academic honesty for assignments for which the student will be graded. Should a situation arise which is not described below, the student is advised to ask the instructor or assistant of the course whether what they intend to do would remain within the framework of academic honesty or not.

a. What is acceptable when preparing an assignment?

- Communicating with classmates about the assignment to understand it better
-
- Putting ideas, quotes, paragraphs, small pieces of code (snippets) that you find online or elsewhere into your assignment, provided that
 - these are not themselves the whole solution to the assignment,
 - you cite the origins of these
-
- Asking sources for help in guiding you for the English language content of your assignment.
 - Sharing small pieces of your assignment in the classroom to create a class discussion on some controversial topics.
-
- Turning to the web or elsewhere for instructions, references, and solutions to technical difficulties, but not for direct answers to the assignment
 - Discuss solutions to assignments with others using diagrams or summarized statements but not actual text or code.
 - Working with (and even paying) a tutor to help you with the course, provided the tutor does not do your assignment for you.
-

b. What is not acceptable?

- Ask a classmate to see their solution to a problem before submitting your own.
 - Failing to cite the origins of any text (or code for programming courses) that you discover outside of the course's lessons and integrate into your work
 - You are giving or showing a classmate your solution to a problem when the classmate is struggling to solve it.
-

J. Expectations

You are expected to attend classes on time by completing weekly course requirements (readings and assignments) during the semester. The main communication channel between the instructor and the students email emailed. Please send your questions to the instructor's email address about the course via the email address provided to you by the university. ***Ensure that you include the course name in the subject field of your message and your name in the text field.*** In addition, the instructor will contact you via email if necessary. For this reason, it is very important to check your email address every day for healthy communication.

K. Lecture Content and Syllabus Updates

If deemed necessary, changes in the lecture content or course schedule can be made. If any changes are made in the scope of this document, the instructor will inform you about this.

Course Schedule Overview

Week	Date	Subjects	Other Tasks
Week 1	13.02.2026	Course Introduction, Software Engineering Fundamentals, What is OOP, Introduction to Java (Classes, Objects, Methods, Method Overloading, Constructors, Access Modifiers, Reusable Technology - OCSF)	
Week 2	20.02.2026	OOP with Java Part-II (Inheritance, Constructors in Inheritance, super/this/instanceof Keywords, Method Overriding, Polymorphism, Encapsulation, final Keyword, Abstract Classes, Object Class)	
Week 3	27.02.2026	OOP with Java Part-III (Interfaces, Nested Interfaces, Extending Interfaces, Modern Interface Features, Abstract Class vs Interface, Nested/Static/Anonymous Classes, Enums, Packages, Reflection, Wrapper Classes, Lambda Expressions)	

Week	Date	Subjects	Other Tasks
Week 4	06.03.2026	UML - Unified Modelling Language (Class Diagrams, Sequence Diagrams, Use Case Diagrams, State Diagrams)	

Week	Date	Subjects	Other Tasks
Week 5	13.03.2026	PlantUML (UML Diagram Generation from Text, Tool Integration, Practical Examples)	
Week 6	20.03.2026	UMPLE Part-1 (Model-Driven Development, UML-to-Code Generation, Basic Syntax and Examples)	
Week 7	27.03.2026	UMPLE Part-2 (Advanced Features, State Machines, Associations, Code Generation). Midterm Project Progress Review	Quiz-1, Project Demonstrations

Week	Date	Subjects	Other Tasks
Week 8	03.04.2026	Midterm Exam Week	Midterm Project Report (MPR1) Due
Week 9	10.04.2026	Design Patterns Introduction & Creational Patterns (What Are Design Patterns, GoF Classification, Factory Method, Abstract Factory, Builder, Prototype, Singleton)	

Week	Date	Subjects	Other Tasks
Week 10	17.04.2026	Structural Design Patterns (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy)	
Week 11	24.04.2026	Behavioral Design Patterns (Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor)	

Week	Date	Subjects	Other Tasks
Week 12	01.05.2026	Code Smells and Refactoring Fundamentals (What/When/How to Refactor, Bloaters, OO Abusers, Change Preventers, Dispensables, Couplers)	
Week 13	08.05.2026	Refactoring Techniques (Composing Methods, Moving Features, Organizing Data, Simplifying Conditionals, Simplifying Method Calls, Dealing with Generalization)	Quiz-2
Week	Date	Subjects	Other Tasks
Week 14	15.05.2026	Case Studies (Applying Design Patterns and Refactoring to Real-World Projects)	
Week 15	22.05.2026	Final Project Progress Review and Summary. Project Demonstrations	Final Project Report (MPR2) Due
Week 16	29.05.2026	Final Exam Week	

End – Of – CEN206 – Syllabus